

NOMBRE DE LA UNIVERSIDAD

MAESTRÍA EN INGENIERÍA DE SOFTWARE

Desarrollo de Aplicaciones Empresariales

Sistema de Reserva de Canchas Deportivas

Arquitectura de Microfrontends y Microservicios

Proyecto Integrador

Autores

Pablo Jara Muñoz

pablojaramunoz@gmail.com

Miguel Lara

miguel.larasj@gmail.com

Sebastián Uyaguari

sebastianuyaguari@gmail.com

Adrián Espinoza

adrian.espinoza1292@gmail.com

Docente

Christian Merchán Millán

Ciudad, 28 de agosto de 2026

Resumen

Se desarrolló un sistema web para la reserva de canchas deportivas de tenis, básquet y pádel, que permite consultar disponibilidad, crear y cancelar reservas, administrar canchas y usuarios, y visualizar reportes básicos. La aplicación se organizó mediante una arquitectura basada en microfrontends y microservicios. En el frontend se implementó un shell junto con tres microfrontends remotos utilizando React y Module Federation, mientras que el backend se dividió en los microservicios ms-usuarios, ms-canchas, ms-reservas y ms-reportes desarrollados con Spring Boot. La persistencia se organizó en PostgreSQL mediante bases de datos independientes para los microservicios que requieren almacenar información, evitando el acceso directo a los datos administrados por otros servicios. También se implementaron las reglas relacionadas con permisos, cancelaciones, estados de reserva, límite de reservas activas y control de solapamiento de horarios. Para este último caso, la validación realizada por ms-reservas se complementó con una restricción de exclusión en PostgreSQL para evitar conflictos ante solicitudes concurrentes. Los componentes del sistema se ejecutan de forma conjunta mediante Docker Compose. Como resultado, se implementaron las funcionalidades definidas para el usuario final y el administrador, y los seis criterios de aceptación establecidos para el proyecto se encuentran cubiertos por la implementación y las pruebas realizadas.

Palabras clave: microservicios, microfrontends, Module Federation, Spring Boot, PostgreSQL, Docker.

Índice

Resumen	i
1. Introducción	1
1.1. Planteamiento del problema	1
1.2. Justificación	2
1.3. Estructura del documento	2
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Alcance funcional	3
3.1. Roles y permisos	4
3.2. Módulos y pantallas	4
3.3. Fuera de alcance	5
4. Arquitectura de la solución	6
4.1. Visión general	6
4.2. Notación	6
4.3. Vistas arquitectónicas	7
4.4. Seguridad y control de acceso	14
4.5. Registro de decisiones de arquitectura	14
5. Modelo de datos	19
5.1. Estrategia de persistencia	19
5.2. Diagrama entidad-relación	20
5.3. Diccionario de datos	20
5.4. Integridad entre bases de datos	24
5.5. Restricciones que implementan reglas de negocio	25

6. Reglas de negocio	26
6.1. Catálogo de reglas	26
6.2. Validación de solapamiento de horarios	28
6.3. Estados de una reserva	29
6.4. Traducción de reglas a respuestas HTTP	30
7. Resultados	32
7.1. Funcionalidades implementadas	32
7.2. Cumplimiento de los criterios de aceptación	33
7.3. Verificación de las reglas de negocio	34
7.4. Limitaciones	36
8. Conclusiones	37
8.1. Lecciones aprendidas	38
8.2. Trabajo futuro	39
Referencias	40
A. Scripts de base de datos	40
B. Colección de pruebas de la API	40
C. Capturas de la aplicación	41
D. Repositorio del proyecto	41

Índice de figuras

1.	Vista de contexto: el sistema y sus dos tipos de usuario.	8
2.	Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.	9
3.	Vista de componentes de ms-reservas	12
4.	Vista de despliegue: contenedores Docker, redes y volumen.	13
5.	Modelo entidad-relación, con la frontera de cada base de datos.	20

Índice de tablas

1.	Matriz de roles y permisos.	4
2.	Módulos y pantallas del sistema.	5
3.	Composición de microfrontends.	10
4.	Composición de microservicios.	11
5.	Reparto de bases de datos.	19
6.	Entidad usuario.	21
7.	Entidad cancha.	21
8.	Entidad bloqueo_mantenimiento.	22
9.	Entidad reserva.	23
10.	Entidad configuracion	24
11.	Restricciones e índices principales del modelo de datos.	25
12.	Reglas de negocio y dónde se implementa cada una.	27
13.	Correspondencia entre violación de regla y respuesta HTTP.	31
14.	Cumplimiento de los criterios de aceptación.	33
15.	Comprobación de las reglas de negocio.	35

Pendientes

1. Introducción

En el presente informe de arquitectura de software se aborda el diseño de un sistema web para la gestión de reservas de canchas deportivas de tenis, básquet y pádel. Esta solución busca centralizar y facilitar la gestión de disponibilidad, el agendamiento y la cancelación de reservas, así como la gestión de canchas, horarios, usuarios y reportes relacionados con su uso.

El sistema se plantea bajo el enfoque de aplicaciones empresariales, considerando tanto la operación diaria del proceso de reservas como la necesidad de mantener una solución organizada, escalable y mantenible. Para ello, se propone una arquitectura basada en microservicios y microfrontends, con una separación clara de responsabilidades entre los módulos funcionales y técnicos.

En el frontend propone utilizar Module Federation para integrar módulos independientes, mientras que en el backend se considera el uso de Spring Boot para el desarrollo de microservicios y PostgreSQL para la persistencia de datos del sistema. Con esta base, cada módulo puede evolucionar de manera más controlada y desplegarse de forma autónoma según las necesidades del sistema.

1.1. Planteamiento del problema

La gestión de reservas de canchas deportivas presenta dificultades cuando se realiza mediante registros manuales o canales no integrados entre sí, como hojas de cálculo, calendarios, pizarras cuadrículadas. En este escenario, la consulta de horarios disponibles es confusa si no se mantiene un orden establecido, el registro o cancelación de reservas puede depender de validaciones manuales y la administración pierde visibilidad oportuna sobre el uso de las canchas y la demanda del servicio.

Esta situación incrementa el riesgo de inconsistencias, como el solapamiento de horarios, la falta de trazabilidad sobre quién realizó una reserva y bajo qué condiciones. Además, limita la capacidad de la organización para gestionar datos personales, aplicar reglas de negocio de manera uniforme y disponer de reportes que apoyen la toma de decisiones.

Frente a este contexto, surge la necesidad de contar con un sistema web centralizado que permita consultar disponibilidad, registrar reservas, gestionar usuarios y canchas, y generar información útil para la operación y el control administrativo.

1.2. Justificación

La solución se aborda con una arquitectura basada en microservicios y microfrontends debido a que el dominio del problema presenta responsabilidades diferenciadas, como la gestión de usuarios, reservas, canchas y reportes. Separar estos componentes facilita su desarrollo, mantenimiento y evolución, evitando concentrar toda la lógica en una sola aplicación de gran tamaño.

Desde el punto de vista del frontend, el uso de microfrontends permite organizar la interfaz por módulos funcionales y mantener cierto nivel de independencia entre ellos, lo que resulta conveniente para incorporar nuevas funcionalidades sin afectar por completo la aplicación principal. En el backend, los microservicios favorecen el aislamiento de reglas de negocio, la autonomía y una mejor alineación entre cada servicio y su responsabilidad dentro del sistema.

En conjunto, esta aproximación responde no solo a una decisión tecnológica, sino también a la necesidad de construir una solución más ordenada, extensible y adecuada para un escenario empresarial donde distintos procesos deben integrarse sin perder claridad ni control.

1.3. Estructura del documento

El documento se organiza en ocho secciones principales. Después de esta introducción, la Sección 2 presenta el objetivo general y los objetivos específicos del proyecto. La Sección 3 define el alcance funcional del sistema, incluyendo los roles, permisos, módulos y funcionalidades que se encuentran fuera del alcance.

La Sección 4 describe la arquitectura del sistema, sus principales vistas y las decisiones arquitectónicas adoptadas. La Sección 5 presenta el modelo de datos, la estrategia de persistencia y las relaciones existentes entre las bases de datos de los microservicios. La Sección 6 detalla las reglas de negocio relacionadas con el proceso de reserva y la forma en que se aplican dentro del sistema.

Posteriormente, la Sección 7 presenta los resultados obtenidos, el cumplimiento de los criterios de aceptación y la verificación de las reglas de negocio. Finalmente, la Sección 8 reúne las conclusiones, las lecciones aprendidas y el trabajo futuro. El documento se complementa con anexos que contienen los scripts de base de datos, la colección de pruebas de la API, capturas de la aplicación y la referencia al repositorio del proyecto.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un sistema web para la reserva de canchas deportivas de tenis, básquet y pádel, que permita consultar disponibilidad, crear y cancelar reservas, gestionar canchas y usuarios, y visualizar reportes básicos, mediante una arquitectura basada en microfrontends y microservicios.

2.2. Objetivos específicos

- Implementar las funcionalidades principales del sistema, de manera que los usuarios puedan consultar disponibilidad, crear y cancelar reservas, mientras que los administradores puedan gestionar canchas y usuarios, cancelar reservas y consultar reportes.
- Diseñar una arquitectura basada en microfrontends que permita separar la interfaz en módulos funcionales independientes e integrarlos dentro de una misma solución web.
- Diseñar una arquitectura de microservicios que distribuya de forma clara las responsabilidades relacionadas con usuarios, canchas, reservas y reportes.
- Definir una estrategia de persistencia que mantenga separados los datos administrados por cada microservicio y evite dependencias directas entre sus modelos de información.
- Implementar las reglas de negocio relacionadas con la disponibilidad, el solapamiento de reservas, las cancelaciones, los estados de reserva y los permisos asociados al rol del usuario.
- Justificar las principales decisiones arquitectónicas y tecnológicas adoptadas en la solución, considerando su impacto en la modularidad, mantenibilidad y separación de responsabilidades, así como su correspondencia con el alcance funcional del sistema.

3. Alcance funcional

El sistema contempla las funcionalidades necesarias para consultar la disponibilidad de las canchas, crear y cancelar reservas, administrar canchas, horarios y usuarios, y visualizar reportes básicos relacionados con su utilización. El alcance se organiza a partir de dos roles principales: usuario final y administrador, cada uno con permisos diferenciados

dentro de la aplicación.

3.1. Roles y permisos

La Tabla 1 presenta las funcionalidades disponibles para cada rol y las restricciones aplicadas en aquellas operaciones que dependen del usuario autenticado.

Tabla 1: Matriz de roles y permisos.

Funcionalidad	Usuario final	Administrador
Registro e inicio de sesión	Sí	Sí
Consultar disponibilidad de canchas	Sí	Sí
Crear una reserva	Sí	No
Cancelar una reserva propia	Sí	No
Cancelar cualquier reserva	No	Sí
Consultar historial de reservas propias	Sí	No
Gestionar catálogo de canchas (crear/editar/inactivar)	No	Sí
Gestionar horarios y bloqueos de mantenimiento	No	Sí
Gestionar usuarios (activar/inactivar)	No	Sí
Visualizar reportes básicos de ocupación	No	Sí

3.2. Módulos y pantallas

Las funcionalidades se agrupan en cuatro módulos principales: seguridad, reservas, administración y reportes. Esta organización permite agrupar las operaciones según su responsabilidad funcional y mantener una separación clara entre autenticación, gestión de reservas, administración del sistema y consulta de información.

Tabla 2: Módulos y pantallas del sistema.

Módulo	Pantalla	Descripción
Seguridad	Inicio de sesión / Registro	Autenticación de usuario final y administrador.
Reservas	Consulta de disponibilidad	Calendario o grilla de horarios disponibles por cancha y deporte.
Reservas	Nueva reserva	Formulario para reservar una cancha, fecha y bloque horario.
Reservas	Mis reservas	Listado de reservas del usuario con opción de cancelar.
Administración	Gestión de canchas	Administración de canchas: nombre, deporte, horario de atención y estado.
Administración	Gestión de reservas	Listado global de reservas con opción de cancelar cualquiera.
Reportes	Reportes básicos	Ocupación por cancha y deporte, reservas por período y cancelaciones.

3.3. Fuera de alcance

Se excluyen las siguientes funcionalidades por decisión explícita:

- Pasarela de pagos o cobro en línea de las reservas.
- Notificaciones automáticas por correo electrónico, SMS o push.
- Reservas recurrentes o recurrencia de horarios, por ejemplo, “todos los lunes”.
- Gestión de torneos, ligas o inscripciones grupales.
- Aplicación móvil nativa, manteniéndose el sistema como una aplicación web responsive.
- Reportes avanzados con inteligencia de negocio (BI) o exportación a formatos analíticos.

4. Arquitectura de la solución

A partir del alcance funcional definido, se plantea la arquitectura de la solución y la forma en que se organizan sus componentes principales. En esta sección se presentan las vistas utilizadas para representar el sistema y se explican las decisiones más importantes relacionadas con la separación de responsabilidades, la comunicación entre componentes, la persistencia y el despliegue.

4.1. Visión general

La arquitectura combina microfrontends en la capa de presentación con una arquitectura de microservicios en el backend.

El frontend se divide en cuatro elementos principales: un shell que actúa como aplicación contenedora y tres microfrontends remotos, mf-reservas, mf-administracion y mf-reportes. Cada microfrontend remoto concentra las funcionalidades asociadas a una responsabilidad específica, mientras que el shell se encarga de integrarlos dentro de una misma aplicación web.

El backend se organiza en cuatro microservicios: ms-usuarios, ms-canchas, ms-reservas y ms-reportes. Cada servicio mantiene las responsabilidades correspondientes a su área funcional. Cuando una operación requiere información administrada por otro servicio, esta se obtiene mediante su API REST, evitando el acceso directo a datos que pertenecen a otro microservicio. Los servicios que requieren persistencia mantienen además su propio modelo de datos.

El acceso al sistema se organiza mediante un Edge implementado con Nginx y un API Gateway. El Edge publica el shell, los microfrontends y las APIs bajo un mismo origen, mientras que el Gateway recibe las solicitudes dirigidas al backend y las enruta hacia el microservicio correspondiente. Internamente, los microservicios siguen una arquitectura hexagonal basada en puertos y adaptadores, separando la lógica de aplicación de los mecanismos utilizados para recibir solicitudes, persistir información o comunicarse con otros servicios.

4.2. Notación

Para representar la arquitectura se utiliza el modelo C4 creado por Simon Brown [1], considerando los niveles de contexto, contenedores y componentes, complementados con una vista de despliegue.

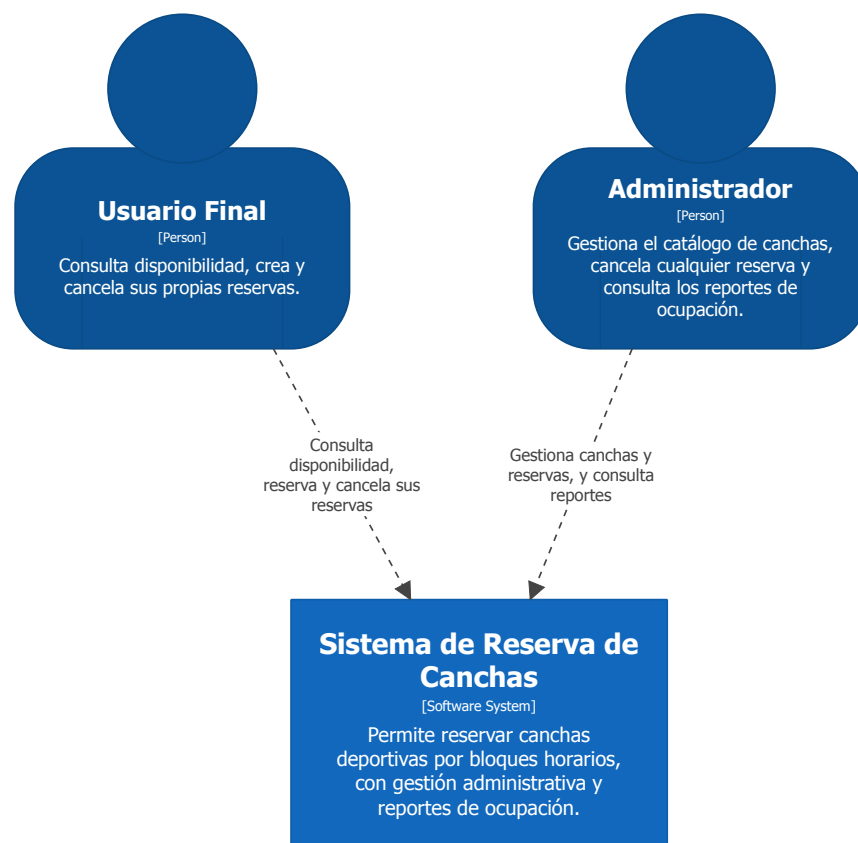
Los diagramas se definen mediante Structurizr DSL en `diagramas/workspace.dsl`,

utilizando un mismo modelo para representar los elementos del sistema y sus relaciones en distintos niveles de detalle. A partir de este modelo se contemplan vistas de contexto, contenedores, componentes y despliegue. En esta sección se presentan las vistas que resultan más útiles para explicar la solución adoptada en el proyecto.

4.3. Vistas arquitectónicas

Las vistas arquitectónicas presentan el sistema desde distintos niveles de detalle. Se parte de una representación general de los usuarios y su relación con el sistema, para posteriormente mostrar la distribución de los contenedores y la estructura interna de los componentes que requieren un mayor nivel de detalle. Finalmente, la vista de despliegue muestra cómo estos elementos se distribuyen en el entorno de ejecución definido para el proyecto.

4.3.1. V-01 Contexto del sistema



System Context View: Sistema de Reserva de Canchas

Nivel 1 — El sistema y sus usuarios.

Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 1: Vista de contexto: el sistema y sus dos tipos de usuario.

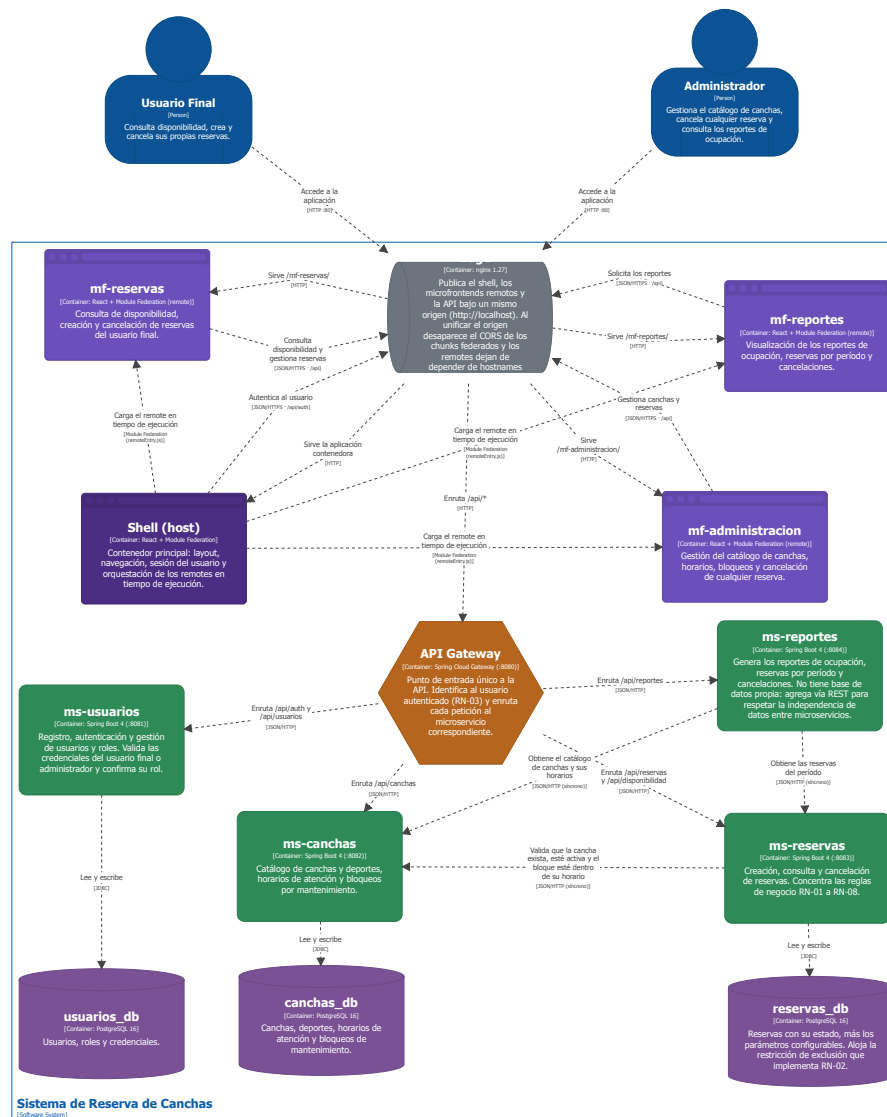
La vista de contexto presenta al sistema como el punto central para la gestión de reservas de canchas deportivas y muestra la relación que mantiene con los dos tipos de usuario definidos.

El usuario final puede registrarse e iniciar sesión, consultar la disponibilidad de las canchas, crear reservas, cancelar sus propias reservas y consultar su historial de reservas.

El administrador puede registrarse e iniciar sesión, consultar la disponibilidad de las canchas, gestionar el catálogo de canchas, administrar horarios y bloqueos de mantenimiento, gestionar usuarios, cancelar cualquier reserva y visualizar reportes básicos de ocupación.

4.3.2. V-02 Contenedores

La vista de contenedores muestra la distribución principal del sistema y la forma en que se relacionan sus elementos de frontend, backend y persistencia. En ella se representan el Edge, el shell y los microfrontends, el API Gateway, los cuatro microservicios y las bases de datos asociadas a los servicios que requieren persistencia.



Container View: Sistema de Reserva de Canchas
 Nivel 2 — Microfrontends, gateway, microservicios y bases de datos.
 Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 2: Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.

Frontend. El frontend se organiza mediante un shell que funciona como host y tres microfrontends remotos. El shell mantiene la estructura general de la aplicación, la navegación y la información de la sesión, y se encarga de cargar los microfrontends

en tiempo de ejecución mediante Module Federation. Cada microfrontend agrupa las pantallas y funcionalidades correspondientes a su área, mientras que el shell permite integrarlos dentro de una misma aplicación web.

Tabla 3: Composición de microfrontends.

Microfrontend	Responsabilidad	Tipo
shell	Gestiona la estructura general, navegación, sesión del usuario y carga de los microfrontends remotos.	Host
mf-reservas	Agrupar la consulta de disponibilidad, creación y cancelación de reservas y el historial de reservas del usuario.	Remote
mf-administracion	Agrupar la gestión de canchas, horarios, bloqueos de mantenimiento, usuarios y la cancelación administrativa de reservas.	Remote
mf-reportes	Agrupar la consulta de reportes de ocupación, reservas por período, cancelaciones y demanda de las canchas.	Remote

Module Federation permite cargar y compartir módulos entre aplicaciones web separadas [3], mecanismo utilizado para integrar los microfrontends remotos en el shell.

Backend. Los microservicios del proyecto se implementan con Spring Boot, cuya documentación oficial cubre aplicaciones web y configuración de almacenes SQL [5].

El backend se organiza mediante un API Gateway y cuatro microservicios. El Gateway funciona como punto de entrada a las APIs del sistema y se encarga de dirigir cada solicitud hacia el microservicio correspondiente. Las reglas de negocio permanecen dentro de los servicios responsables de cada área funcional.

Tabla 4: Composición de microservicios.

Microservicio	Responsabilidad	Base de datos
ms-usuarios	Gestiona el registro, autenticación, usuarios y roles del sistema.	usuarios_db
ms-canchas	Gestiona el catálogo de canchas, deportes, horarios de atención y bloqueos de mantenimiento.	canchas_db
ms-reservas	Gestiona la consulta de disponibilidad, creación, consulta y cancelación de reservas.	reservas_db
ms-reportes	Genera los reportes a partir de la información obtenida de los servicios de reservas y canchas.	Ninguna

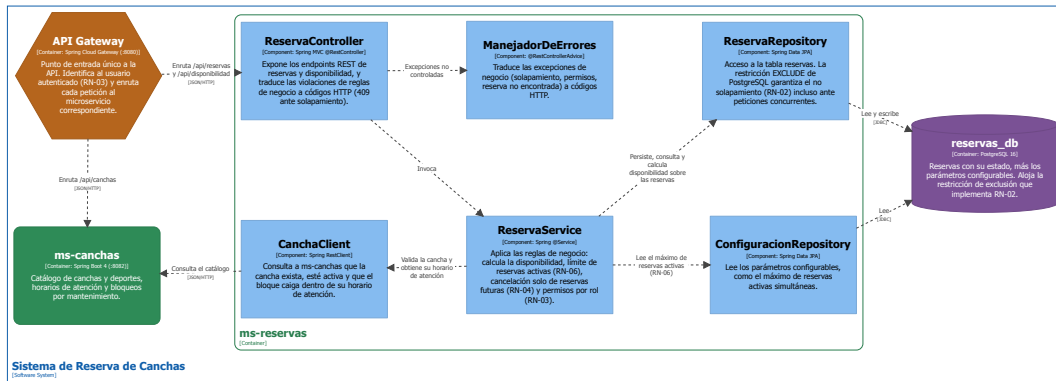
Comunicación. La comunicación entre los contenedores del backend se realiza mediante HTTP/REST utilizando mensajes JSON. Las solicitudes provenientes del frontend ingresan a través del API Gateway, que las dirige hacia ms-usuarios, ms-canchas, ms-reservas o ms-reportes según la ruta solicitada.

También existen comunicaciones entre microservicios, por ejemplo, ms-reservas consulta de forma síncrona a ms-canchas para obtener información de la cancha y sus horarios. Por su parte, ms-reportes consulta a ms-reservas y ms-canchas para obtener los datos necesarios para generar los reportes.

Al utilizar comunicación síncrona, una operación que depende de otro microservicio no puede continuar si el servicio requerido no se encuentra disponible, en estos casos, el error se devuelve mediante una respuesta HTTP indicando que la solicitud no pudo completarse.

4.3.3. V-03 Componentes de ms-reservas

Se detalla a profundidad **ms-reservas** debido a que en este servicio se concentran las principales reglas sobre la disponibilidad, creación y cancelación de reservas. El diagrama muestra cómo se organiza internamente el servicio y cómo se separa la lógica de negocio de los mecanismos utilizados para recibir solicitudes, consultar otros servicios y acceder a la base de datos.



Component View: Sistema de Reserva de Canchas - ms-reservas
 Nivel 3 - Interior de ms-reservas, donde se aplican las reglas de negocio.
 Sunday, August 23, 2025 at 10:04 PM Ecuador Time

Figura 3: Vista de componentes de ms-reservas.

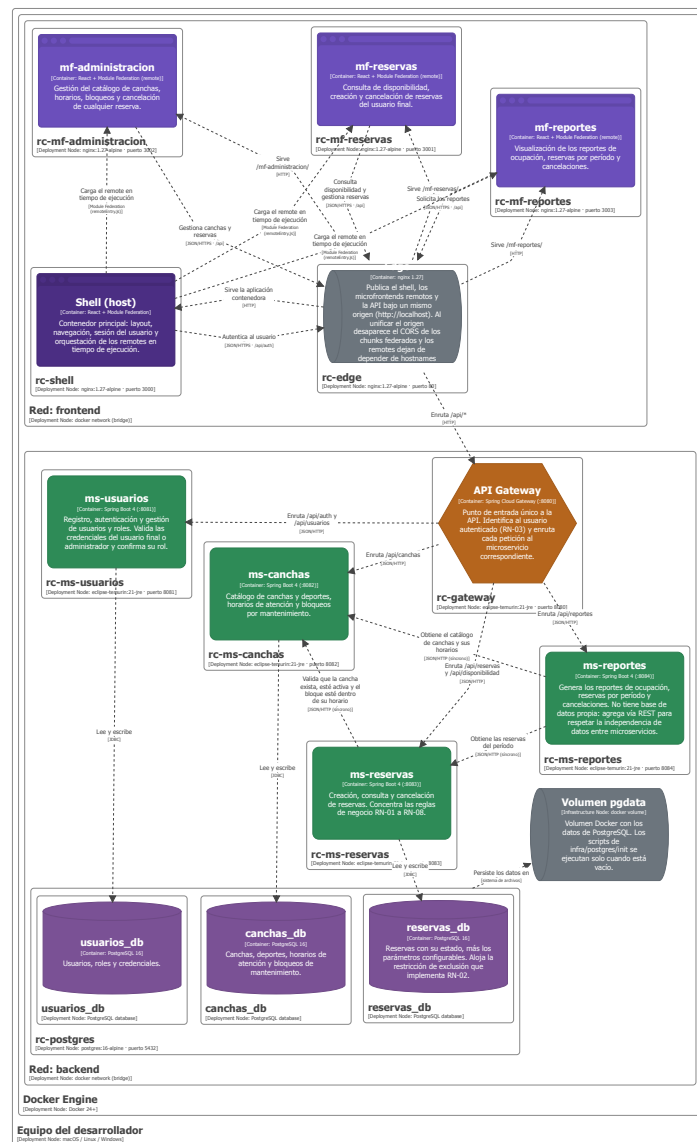
Cuando se crea una reserva, la solicitud llega desde el API Gateway a BookingController, que funciona como punto de entrada de ms-reservas. Desde el controlador se utiliza BookingUseCase para ejecutar la operación, mientras que BookingService se encarga de aplicar las validaciones y reglas relacionadas con la reserva.

Durante este proceso, BookingService utiliza diferentes puertos según la información que necesita. CourtClientPort permite consultar a ms-canchas para validar la cancha y obtener sus horarios. ConfigurationRepositoryPort permite consultar parámetros como el número máximo de reservas activas, mientras que BookingRepositoryPort se utiliza para consultar y registrar las reservas.

Cada uno de estos puertos cuenta con un adaptador encargado de realizar la operación correspondiente. CourtClientAdapter realiza la comunicación REST con ms-canchas, mientras que los adaptadores de persistencia se encargan del acceso a reservas_db. Esta separación permite que la lógica de las reservas se mantenga independiente de la forma en que se consultan otros servicios o se almacenan los datos.

4.3.4. V-04 Despliegue

Docker Compose permite definir y ejecutar aplicaciones compuestas por varios contenedores desde una configuración única [2]. PostgreSQL se utiliza como motor de persistencia para los servicios que almacenan datos [4].



Deployment View: Sistema de Reserva de Canchas - Local (Docker Compose)
Materialización del sistema en contenedores Docker.
Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 4: Vista de despliegue: contenedores Docker, redes y volumen.

El sistema se plantea para desplegarse localmente mediante Docker Compose, de modo que sus componentes principales puedan ejecutarse en contenedores independientes. Dentro de esta configuración se consideran dos redes de tipo bridge: frontend y backend. La red frontend agrupa el shell y los tres microfrontends, mientras que la red backend contiene el API Gateway, los microservicios y la instancia de PostgreSQL.

El Edge se ubica como punto de acceso común entre ambas redes, lo que permite mantener separados los componentes de presentación y los servicios internos del backend. Bajo este esquema, el acceso principal a la aplicación se realiza a través del Edge y del API Gateway.

PostgreSQL se considera como una única instancia que aloja las bases de datos `usuarios_db`, `canchas_db` y `reservas_db`. De esta manera, cada microservicio mantiene la separación lógica de los datos que administra, mientras que `ms-reportes` no requiere una base de datos propia para los reportes contemplados en el alcance actual.

4.4. Seguridad y control de acceso

La autenticación del sistema se realiza mediante HTTP Basic. Dentro de este esquema, `ms-usuarios` es el servicio responsable de validar las credenciales y mantener la información de usuarios y roles.

En cada solicitud autenticada, el cliente envía la cabecera `Authorization` utilizando el esquema Basic. El API Gateway verifica esa credencial antes de permitir que la petición continúe hacia el microservicio correspondiente.

Una vez validada la identidad, el Gateway incorpora las cabeceras `X-User-Id` y `X-User-Role` en la solicitud que se reenvía al backend. De esta manera, cada microservicio recibe la identidad y el rol del usuario y puede aplicar las reglas de autorización que correspondan.

Además, el Gateway debe descartar cualquier cabecera `X-User-*` recibida directamente desde el cliente, evitando que la identidad del usuario pueda ser enviada sin haber sido validada previamente.

4.5. Registro de decisiones de arquitectura

Las principales decisiones tomadas durante el diseño se registran con el fin de dejar claro el problema que motivó cada una, el enfoque adoptado y las consecuencias que genera dentro de la arquitectura. A continuación, se presenta un resumen de las decisiones aceptadas para el sistema.

ADR-01 Arquitectura de microfrontends con Module Federation

Contexto. La interfaz reúne funcionalidades diferenciadas para reservas, administración y reportes. Mantenerlas dentro de una única aplicación aumentaría la dependencia entre los módulos y dificultaría mantener una separación clara entre las áreas funcionales.

Decisión. Se adopta una arquitectura de microfrontends utilizando Module Federation. El frontend se organiza mediante un shell que funciona como host y los remotes `mf-reservas`, `mf-administracion` y `mf-reportes`.

Alternativa descartada. Se consideró mantener las funcionalidades dentro de

una única aplicación, pero esto incrementaría la dependencia entre los distintos módulos funcionales.

Consecuencia. Se obtiene una mayor separación entre los módulos del frontend, aunque se incorpora complejidad adicional en la integración de los remotes, las dependencias compartidas y su coordinación.

ADR-02 Arquitectura de microservicios para el backend

Contexto. El sistema contiene responsabilidades diferentes relacionadas con usuarios, canchas, reservas y reportes, cada una con reglas y modelos de información propios.

Decisión. Se adopta una arquitectura basada en microservicios desarrollados con Spring Boot. El backend se organiza en ms-usuarios, ms-canchas, ms-reservas y ms-reportes, exponiendo sus funcionalidades mediante APIs REST.

Alternativa descartada. Se consideró concentrar toda la lógica del backend dentro de una única unidad, pero esto dificultaría mantener límites claros entre las distintas responsabilidades.

Consecuencia. La lógica correspondiente a cada área se mantiene separada, aunque las operaciones que requieren información de otros servicios necesitan mecanismos explícitos de comunicación e integración.

ADR-03 Independencia de datos entre microservicios

Contexto. El acceso directo de un microservicio a las tablas administradas por otro generaría dependencia sobre estructuras de datos ajenas y reduciría la autonomía de los servicios.

Decisión. Cada microservicio responsable de persistencia administra su propio modelo de datos. ms-usuarios administra usuarios_db, ms-canchas administra canchas_db y ms-reservas administra reservas_db. La información perteneciente a otro servicio se obtiene mediante su API.

Alternativa descartada. Se descarta el acceso directo a las tablas o bases de datos administradas por otros microservicios.

Consecuencia. Los modelos de persistencia permanecen separados, aunque determinadas operaciones requieren comunicación adicional mediante HTTP/REST.

ADR-04 Instancia PostgreSQL compartida con bases independientes

Contexto. Se requiere mantener la independencia de los datos administrados por los microservicios sin introducir una infraestructura innecesariamente compleja para

el entorno de ejecución del proyecto.

Decisión. Se utiliza una única instancia de PostgreSQL que aloja tres bases de datos independientes: usuarios_db, canchas_db y reservas_db. Cada microservicio dispone únicamente de acceso a la base correspondiente a su responsabilidad.

Alternativa descartada. Se consideró utilizar una instancia independiente de PostgreSQL para cada microservicio, pero esto incrementaría la infraestructura necesaria para ejecutar el sistema.

Consecuencia. Se conserva la separación lógica de los datos con una infraestructura más simple, aunque una falla en la instancia de PostgreSQL puede afectar simultáneamente a los servicios que dependen de ella.

ADR-05 Generación de reportes mediante composición de servicios REST

Contexto. La generación de reportes requiere combinar información administrada por ms-reservas y ms-canchas. El acceso directo a sus bases de datos rompería la independencia establecida entre los servicios.

Decisión. ms-reportes no dispone de una base de datos propia para los reportes actuales. La información se obtiene mediante llamadas REST síncronas a ms-reservas y ms-canchas, y posteriormente se combinan los datos para calcular los indicadores requeridos.

Alternativa descartada. Se descarta que ms-reportes consulte directamente reservas_db y canchas_db.

Consecuencia. Se mantiene la propiedad de los datos dentro de los microservicios que los administran, aunque la generación de reportes depende de la disponibilidad de ms-reservas y ms-canchas.

ADR-06 React para la implementación de los microfrontends

Contexto. La capa de presentación se distribuye entre un shell y varios microfrontends remotos, por lo que se requiere una tecnología que permita construir interfaces basadas en componentes y mantener una estructura común entre ellos.

Decisión. Se utiliza React para implementar el shell, mf-reservas, mf-administracion y mf-reportes. La integración se realiza mediante Module Federation y los remotes se cargan en tiempo de ejecución.

Alternativa descartada. El ADR no registra una alternativa tecnológica específica. La decisión se centra en utilizar React como tecnología común para el shell y los microfrontends.

Consecuencia. Se mantiene un enfoque homogéneo en la capa de presentación,

aunque deben coordinarse las versiones y la configuración de las dependencias compartidas para evitar duplicaciones o incompatibilidades.

ADR-07 API Gateway como punto único de acceso al backend

Contexto. Permitir que cada microfrontend conozca directamente la ubicación y las rutas internas de los microservicios aumentaría el acoplamiento entre el frontend y la estructura de despliegue del backend.

Decisión. Se incorpora un API Gateway implementado con Spring Cloud Gateway como punto único de entrada a las APIs. Las solicitudes recibidas sobre `/api/*` se dirigen hacia `ms-usuarios`, `ms-canchas`, `ms-reservas` o `ms-reportes` según la operación solicitada.

Alternativa descartada. Se descarta que cada microfrontend conozca y se comunique directamente con los distintos microservicios.

Consecuencia. El frontend utiliza un punto de acceso común al backend, aunque el Gateway introduce un salto adicional y una falla en este componente puede impedir temporalmente el acceso a los microservicios desde la aplicación web.

ADR-08 Edge Nginx para unificar el origen de acceso

Contexto. El shell, los microfrontends y las APIs se ejecutan en contenedores diferentes. Exponer cada elemento directamente obligaría al navegador a trabajar con distintas direcciones y puertos.

Decisión. Se incorpora un Edge implementado con Nginx que publica el shell, los microfrontends remotos y las APIs bajo un mismo origen. Las solicitudes asociadas a `/api/*` son dirigidas hacia el API Gateway.

Alternativa descartada. Se considera exponer cada elemento directamente mediante diferentes direcciones y puertos, lo que requeriría manejar solicitudes entre distintos orígenes desde el navegador.

Consecuencia. El acceso desde el navegador se mantiene bajo un origen común, aunque se incorpora un componente adicional cuya configuración debe mantenerse alineada con las rutas del frontend y del API Gateway.

ADR-09 Control de solapamiento de reservas mediante PostgreSQL

Contexto. Validar el solapamiento únicamente desde la lógica de aplicación puede generar una condición de carrera cuando dos solicitudes intentan reservar al mismo tiempo una misma cancha y período.

Decisión. Además de la validación realizada en `ms-reservas`, se utiliza una restricción

de exclusión en PostgreSQL dentro de reservas_db para impedir que se persistan reservas solapadas sobre una misma cancha.

Alternativa descartada. Se descarta depender únicamente de la validación previa realizada desde la lógica de aplicación.

Consecuencia. La regla de no solapamiento se mantiene incluso ante solicitudes concurrentes, aunque parte de su protección depende de una capacidad específica de PostgreSQL y los conflictos rechazados por la base deben ser traducidos por ms-reservas.

ADR-10 Autenticación HTTP Basic validada por el API Gateway

Contexto. El sistema requiere identificar al usuario y conocer su rol durante las solicitudes dirigidas a los diferentes microservicios, sin mantener una sesión independiente en cada uno de ellos.

Decisión. Se define autenticación HTTP Basic como mecanismo de identificación de usuarios. ms-usuarios se encarga de validar las credenciales y, en las solicitudes posteriores, el API Gateway verifica esta información antes de propagar X-User-Id y X-User-Role al microservicio correspondiente.

Alternativa descartada. Se descarta mantener una sesión independiente en cada microservicio, ya que esto obligaría a compartir información de sesión entre los componentes del backend.

Consecuencia. La autenticación se centraliza en el API Gateway, aunque este debe validar correctamente las credenciales en cada petición y descartar cualquier cabecera X-User-* proporcionada directamente por el cliente.

ADR-11 Arquitectura hexagonal para la organización interna de los microservicios

Contexto. Los microservicios deben mantener separada la lógica propia de cada dominio de los mecanismos utilizados para exponer funcionalidades, acceder a la persistencia o comunicarse con otros servicios.

Decisión. Se adopta una arquitectura hexagonal basada en puertos y adaptadores. Los casos de uso y servicios de aplicación se mantienen separados de los adaptadores utilizados para recibir solicitudes, acceder a PostgreSQL o realizar llamadas REST.

Alternativa descartada. Se considera una organización tradicional basada únicamente en controlador, servicio y repositorio, donde la lógica de aplicación mantiene una relación más directa con los detalles tecnológicos.

Consecuencia. La lógica de aplicación queda menos acoplada a la infraestructura y permite modificar los adaptadores sin alterar directamente los casos de uso, aunque se

incrementa la cantidad de interfaces y componentes internos que deben mantenerse.

5. Modelo de datos

5.1. Estrategia de persistencia

La persistencia se organiza según la responsabilidad de cada microservicio. ms-usuarios administra usuarios_db, ms-canchas administra canchas_db y ms-reservas administra reservas_db. Cada servicio accede únicamente a la base de datos que le corresponde y no consulta directamente las tablas administradas por otros microservicios.

Las tres bases de datos se alojan en una misma instancia de PostgreSQL, pero se mantienen separadas mediante bases independientes y credenciales propias para cada servicio. De esta manera, se conserva la separación de los datos sin utilizar una instancia distinta de PostgreSQL por microservicio, lo que simplifica la infraestructura necesaria para el entorno del proyecto. Como consecuencia, una falla en la instancia de PostgreSQL puede afectar a los tres servicios que dependen de ella.

Los esquemas de usuarios_db, canchas_db y reservas_db se gestionan mediante Flyway dentro de sus respectivos microservicios. Esto permite que cada servicio mantenga sus propias migraciones y cambios de estructura. ms-reportes no dispone de una base de datos, ya que obtiene la información necesaria mediante las APIs de ms-reservas y ms-canchas.

Tabla 5: Reparto de bases de datos.

Base	Servicio dueño	Contenido
usuarios_db	ms-usuarios	Usuarios, credenciales, roles y estado de las cuentas.
canchas_db	ms-canchas	Canchas, deportes, horarios de atención y bloqueos de mantenimiento.
reservas_db	ms-reservas	Reservas, estados y parámetros de configuración relacionados con el proceso de reserva.

5.2. Diagrama entidad-relación

El modelo de datos se presenta mediante un único diagrama entidad-relación, agrupando las tablas según la base de datos a la que pertenecen. De esta forma, se pueden observar en conjunto las fronteras de usuarios_db, canchas_db y reservas_db, así como distinguir las relaciones internas de las referencias que cruzan entre bases de datos.

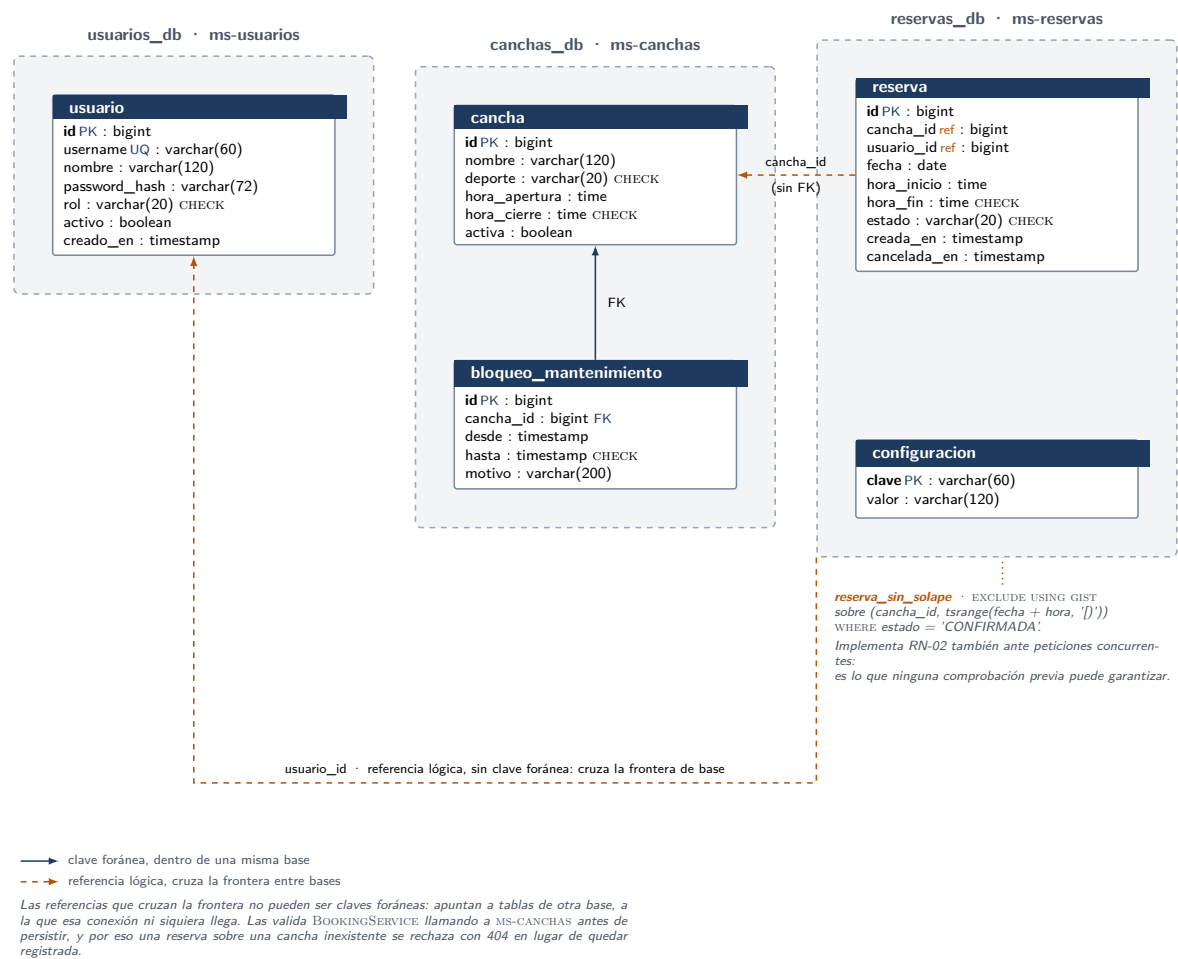


Figura 5: Modelo entidad-relación, con la frontera de cada base de datos.

5.3. Diccionario de datos

El diccionario de datos se organiza según las tablas que forman parte de cada base de datos. usuarios_db contiene la tabla usuario; canchas_db contiene cancha y bloqueo_mantenimiento; mientras que reservas_db contiene reserva y configuracion.

Base usuarios_db

Tabla 6: Entidad usuario.

Campo	Tipo lógico	Tipo SQL	Descripción
id	Identificador	bigint	Identificador de cuenta de usuario.
username	Texto	varchar(60)	Nombre con el que el usuario es reconocido en el sistema.
nombre	Texto	varchar(120)	Nombre visible que identifica a la persona dueña de la cuenta.
password_hash	Texto	varchar(72)	Hash BCrypt de la contraseña del usuario.
rol	Enumeración	varchar(20)	Determina los permisos y las secciones que puede visualizar en el sistema.
activo	Booleano	boolean	Indica si la cuenta puede acceder a la aplicación.
creado_en	Fecha y hora	timestamp	Fecha y hora en que se registró la cuenta.

Base canchas_db

Tabla 7: Entidad cancha.

Campo	Tipo lógico	Tipo SQL	Descripción
id	Identificador	bigint	Identificador de la cancha dentro del catálogo.

Continúa en la siguiente página

Continuación de la Tabla 7

Campo	Tipo lógico	Tipo SQL	Descripción
nombre	Texto	<code>varchar(120)</code>	Nombre con el que la cancha se muestra al usuario.
deporte	Enumeración	<code>varchar(20)</code>	Deporte que se practica en la cancha y con el que se clasifica en el catálogo.
hora_apertura	Hora	<code>time</code>	Hora desde la cual la cancha puede ofrecer bloques de reserva.
hora_cierre	Hora	<code>time</code>	Hora hasta la cual la cancha puede ofrecer bloques de reserva.
activa	Booleano	<code>boolean</code>	Indica si la cancha se encuentra disponible para las reservas.

Tabla 8: Entidad bloqueo_mantenimiento.

Campo	Tipo lógico	Tipo SQL	Descripción
id	Identificador	<code>bigint</code>	Identificador del período de mantenimiento.
cancha_id	Referencia a cancha	<code>bigint</code>	Cancha a la que pertenece el período de mantenimiento.
desde	Fecha y hora	<code>timestamp</code>	Fecha y hora de inicio del mantenimiento.
hasta	Fecha y hora	<code>timestamp</code>	Fecha y hora de finalización del mantenimiento.

Continúa en la siguiente página

Continuación de la Tabla 8

Campo	Tipo lógico	Tipo SQL	Descripción
motivo	Texto	varchar(200)	Motivo del mantenimiento.

Base reservas_db**Tabla 9:** Entidad reserva.

Campo	Tipo lógico	Tipo SQL	Descripción
id	Identificador	bigint	Identificador de la reserva registrada.
cancha_id	Referencia lógica a cancha	bigint	Identificador de la cancha reservada.
usuario_id	Referencia lógica a usuario	bigint	Identificador del usuario que realizó la reserva.
fecha	Fecha	date	Fecha calendario de reservación de la cancha.
hora_inicio	Hora	time	Hora de inicio de la reservación.
hora_fin	Hora	time	Hora de finalización de la reservación.
estado	Enumeración	varchar(20)	Estado de la reserva, determina si está confirmada, cancelada o finalizada.
creada_en	Fecha y hora	timestamp	Fecha y hora en que se registró la reserva.

Continúa en la siguiente página

Continuación de la Tabla 9

Campo	Tipo lógico	Tipo SQL	Descripción
cancelada_en	Fecha y hora	timestamp	Fecha y hora en que se anuló la reserva.

Tabla 10: Entidad configuracion.

Campo	Tipo lógico	Tipo SQL	Descripción
clave	Clave de configuración	varchar(60)	Nombre que identifica el parámetro de configuración.
valor	Valor de la configuración	varchar(120)	Valor asignado al parámetro de configuración.

5.4. Integridad entre bases de datos

La tabla reserva contiene dos referencias hacia información de otros microservicios: cancha_id y usuario_id. Como estos datos pertenecen a canchas_db y usuarios_db, no se utilizan claves foráneas entre estas bases. La validación se realiza antes de guardar la reserva, utilizando la información que proporciona el servicio responsable de cada dato.

- **Cancha.** BookingService utiliza CourtClientPort para consultar a ms-canchas antes de registrar la reserva, para eso se comprueba que la cancha exista, esté activa, que el bloque de tiempo solicitado se encuentre dentro de su horario y que no exista un bloqueo de mantenimiento para ese período. Si la cancha no existe, la operación responde con 404. Si existe pero no puede aceptar la reserva, por ejemplo porque está inactiva, fuera de horario o bloqueada por mantenimiento, se responde con 422. Si ms-canchas no se encuentra disponible, la reserva no puede continuar y se responde con 503.
- **Usuario.** ms-reservas no vuelve a consultar a ms-usuarios para comprobar usuario_id. En el flujo normal de la aplicación, el API Gateway valida primero las credenciales contra ms-usuarios y luego agrega X-User-Id y X-User-Role a la

solicitud. Además, descarta cualquier cabecera X-User-* enviada por el cliente antes de generar estas cabeceras. Por esta razón, ms-reservas utiliza la identidad recibida desde el Gateway para registrar la reserva y aplicar las reglas de acceso correspondientes.

Las claves foráneas sí se utilizan cuando las tablas pertenecen a la misma base de datos. Por ejemplo, bloqueo_mantenimiento.cancha_id referencia a cancha.id dentro de canchas_db. Por lo tanto, las relaciones internas se validan directamente en PostgreSQL, mientras que las referencias hacia otros microservicios se validan antes de persistir la información.

5.5. Restricciones que implementan reglas de negocio

La principal restricción del modelo se encuentra en la tabla reserva y se utiliza para evitar que se guarden reservas solapadas para una misma cancha. Aunque ms-reservas comprueba la disponibilidad antes de registrar una reserva, esta validación por sí sola no cubre el caso en que dos solicitudes intentan reservar el mismo horario al mismo tiempo. Por esta razón, la comprobación final también se mantiene en PostgreSQL.

La restricción reserva_sin_solape utiliza EXCLUDE USING gist sobre cancha_id y el rango formado por fecha, hora_inicio y hora_fin. La comparación se aplica únicamente a las reservas con estado CONFIRMADA. De esta manera, PostgreSQL rechaza una operación si al momento de guardar encuentra otro rango que se solapa para la misma cancha. La restricción requiere la extensión btree_gist para poder utilizar la comparación por igualdad de cancha_id dentro del índice GiST. El funcionamiento completo de esta regla se explica en la Apartado 6.2.

Además de esta restricción, los esquemas contienen validaciones CHECK e índices que mantienen valores válidos y apoyan las consultas realizadas por los microservicios.

Tabla 11: Restricciones e índices principales del modelo de datos.

Tabla	Restricción o índice	Propósito
usuario	usuario_username_unico	Evita que se registren dos usuarios con el mismo username.
usuario	usuario_rol_valido	Limita el rol a USUARIO_FINAL o ADMINISTRADOR.

Continúa en la siguiente página

Continuación de la Tabla 11

Tabla	Restricción o índice	Propósito
cancha	cancha_deporte_valido	Limita el deporte a PADEL, TENIS o BASQUET.
cancha	cancha_horario_valido	Impide guardar una hora de cierre menor o igual a la hora de apertura.
bloqueo_mantenimiento	bloqueo_rango_valido	Comprueba que el final del bloqueo por mantenimiento sea posterior a su inicio.
bloqueo_mantenimiento	bloqueo_por_cancha	Apoya la búsqueda de bloqueos por mantenimiento registrados para una cancha.
reserva	reserva_estado_valido	Limita el estado a CONFIRMADA, CANCELADA o FINALIZADA.
reserva	reserva_bloque_valido	Comprueba que la hora de finalización sea posterior a la hora de inicio.
reserva	reserva_sin_solape	Evita el solapamiento de reservas confirmadas para una misma cancha, incluso cuando se reciben solicitudes concurrentes.
reserva	reserva_por_usuario	Apoya las consultas de reservas por usuario y fecha.
reserva	reserva_por_cancha_fecha	Apoya las consultas de disponibilidad por cancha y fecha.

6. Reglas de negocio

6.1. Catálogo de reglas

Tabla 12: Reglas de negocio y dónde se implementa cada una.

ID	Descripción	Implementación
RN-01	Una reserva se realiza sobre una cancha, una fecha y un bloque horario predefinido de una hora.	<code>ms-reservas:</code> <code>BookingService.crear</code> crea el bloque mediante <code>TimeSlot.deUnaHoraDesde</code> . Prueba: <code>BookingServiceCreacionTest</code> .
RN-02	No pueden existir dos reservas confirmadas que se solapen para la misma cancha.	<code>ms-reservas:</code> <code>BookingService.crear</code> realiza la comprobación previa y <code>BookingRepositoryAdapter</code> maneja el conflicto al persistir. <code>reservas_db</code> : restricción <code>reserva_sin_solape</code> . Pruebas: <code>BookingServiceCreacionTest</code> y <code>ConcurrenciaReservaIT</code> .
RN-03	El usuario final solo puede cancelar sus propias reservas; el administrador puede cancelar cualquiera.	<code>ms-reservas:</code> <code>BookingService.cancelar</code> utiliza <code>X-User-Id</code> para comprobar el dueño de la reserva y <code>X-User-Role</code> para identificar al administrador. Prueba: <code>BookingServiceCancelacionTest</code> .
RN-04	Una reserva no puede cancelarse cuando su fecha y hora de inicio ya ocurrieron, incluso si la operación la realiza un administrador.	<code>ms-reservas:</code> <code>BookingService.cancelar</code> utiliza <code>Booking.sePuedeCancelarEn</code> . Prueba: <code>BookingServiceCancelacionTest</code> .
RN-05	Al cancelar una reserva, su estado cambia a CANCELADA y el bloque vuelve a quedar disponible sin eliminar el registro.	<code>ms-reservas:</code> <code>BookingService.cancelar</code> invoca <code>Booking.cancelar</code> . La restricción <code>reserva_sin_solape</code> solo considera reservas confirmadas. Prueba: <code>BookingServiceCancelacionTest</code> .

Continúa en la siguiente página

Continuación de la Tabla 12

ID	Descripción	Implementación
RN-06	Un usuario final tiene un límite configurable de reservas activas simultáneas, con un valor de 3 por defecto.	ms-reservas: <code>BookingService.verificarTope</code> obtiene el valor mediante <code>ConfigurationRepositoryPort</code> . Prueba: <code>BookingServiceCreacionTest</code> .
RN-07	Solo el administrador puede crear, editar o inactivar canchas y gestionar sus horarios y bloqueos de mantenimiento.	ms-canchas: <code>CourtService</code> aplica <code>exigirAdministrador</code> en las operaciones de gestión. Prueba: <code>CourtServiceTest</code> .
RN-08	Una reserva puede encontrarse en estado CONFIRMADA, CANCELADA o FINALIZADA.	ms-reservas: <code>Booking</code> nace como CONFIRMADA, <code>Booking.cancelar</code> cambia el estado a CANCELADA y <code>Booking.estadoVigente</code> determina cuándo corresponde FINALIZADA. Pruebas: <code>BookingServiceCreacionTest</code> y <code>BookingServiceCancelacionTest</code> .

6.2. Validación de solapamiento de horarios

La **RN-02** establece que una misma cancha no puede tener dos reservas confirmadas que se solapen en el mismo horario. El principal problema aparece cuando dos usuarios intentan reservar el mismo bloque casi al mismo tiempo. En ese caso, ambas solicitudes podrían comprobar la disponibilidad antes de que alguna de las dos haya sido guardada y las dos podrían encontrar el horario libre.

Para evitar esta situación, la regla se controla en dos momentos. Primero, `BookingService.crear` comprueba si el bloque solicitado se solapa con una reserva confirmada existente. Para realizar esta comparación se utiliza `TimeSlot.seSolapaCon`. Esta validación permite detectar el conflicto antes de intentar guardar la reserva en los casos normales.

Sin embargo, esta comprobación no es suficiente cuando dos solicitudes se procesan de forma concurrente. Las dos podrían realizar la validación al mismo tiempo y obtener como resultado que el bloque está disponible. Por esta razón, la comprobación final se mantiene también en PostgreSQL mediante la restricción `reserva_sin_solape`.

```
1 ALTER TABLE reserva ADD CONSTRAINT reserva_sin_solape
```

```
2      EXCLUDE USING gist (  
3          cancha_id WITH =,  
4          tsrange(fecha + hora_inicio, fecha + hora_fin, '[') WITH &&  
5      ) WHERE (estado = 'CONFIRMADA');
```

Listing 1: Restricción que protege la **RN-02** en `V1__reservas.sql`.

La restricción compara el identificador de la cancha y el rango horario de la reserva. Si ya existe una reserva confirmada para la misma cancha cuyo horario se solapa con el nuevo, PostgreSQL rechaza la operación. Así se evita que dos solicitudes concurrentes terminen reservando el mismo bloque aunque ambas hayan superado la comprobación previa de `BookingService`.

La definición también incluye dos condiciones importantes para el funcionamiento de las reservas:

- `WHERE (estado = 'CONFIRMADA')` hace que únicamente las reservas confirmadas ocupen un bloque. Cuando una reserva cambia a `CANCELADA`, deja de participar en esta restricción y el horario vuelve a quedar disponible sin eliminar el registro.
- El rango `'['` incluye la hora de inicio y excluye la hora de finalización. Esto permite, por ejemplo, registrar una reserva de 10:00 a 11:00 y otra de 11:00 a 12:00 para la misma cancha, ya que son bloques consecutivos y no existe solapamiento entre ellos.

La restricción requiere la extensión `btree_gist`, ya que `cancha_id` debe compararse por igualdad dentro del índice GiST. Este requisito se prepara en `infra/postgres/init/02-extensiones.sql`, antes de aplicar la migración `V1__reservas.sql`.

Si el conflicto se detecta al momento de guardar, `BookingRepositoryAdapter` traduce el error de persistencia a `SlotAlreadyBookedException`. La misma excepción se utiliza cuando el solapamiento se detecta previamente en `BookingService`. De esta forma, el conflicto se maneja de la misma manera sin importar en cuál de las dos validaciones haya sido detectado, y la API responde con HTTP 409.

Este caso también se comprueba mediante `ConcurrenciaReservaIT`, donde se intentan registrar dos reservas sobre el mismo bloque de forma concurrente. La prueba verifica que solamente una de las operaciones consiga registrar la reserva y que la otra sea rechazada por solapamiento.

6.3. Estados de una reserva

La **RN-08** establece tres estados para una reserva: `CONFIRMADA`, `CANCELADA` y `FINALIZADA`. Toda reserva se crea inicialmente en estado `CONFIRMADA`.

Una reserva confirmada puede cambiar a **CANCELADA** cuando se realiza una cancelación antes de la hora de inicio. Esta operación se realiza mediante `Booking.cancelar`, que modifica el estado y registra la fecha y hora en que se produjo la cancelación. Una vez cancelada, la reserva permanece en ese estado.

El estado **FINALIZADA** se obtiene de una forma diferente. La reserva no se actualiza en la base de datos cuando termina su horario. Al consultarla, `Booking.estadoVigente` comprueba si continúa confirmada y si la hora de finalización del bloque ya ocurrió. Cuando esto sucede, la reserva se presenta como **FINALIZADA**.

Mientras una reserva ya comenzó pero todavía no termina, continúa apareciendo como **CONFIRMADA**. Sin embargo, ya no puede cancelarse debido a la **RN-04**.

Por lo tanto, las transiciones que maneja la aplicación son las siguientes:

- **CONFIRMADA** → **CANCELADA**, cuando se cancela antes de la hora de inicio.
- **CONFIRMADA** → **FINALIZADA**, cuando el bloque ya terminó y el estado se calcula al consultar la reserva.

No existen transiciones desde **CANCELADA** ni desde **FINALIZADA** hacia otro estado.

6.4. Traducción de reglas a respuestas HTTP

Las reglas de negocio se mantienen separadas de los códigos HTTP. Cuando una operación no puede realizarse, la lógica de la aplicación genera una excepción y el `ErrorHandler` del microservicio correspondiente se encarga de convertirla en una respuesta HTTP. De esta manera, las clases del dominio no necesitan conocer detalles propios de la capa web.

En **ms-reservas**, los conflictos relacionados con el estado actual de una reserva se responden principalmente con **409 Conflict**. En cambio, cuando una cancha existe pero no puede aceptar la reserva solicitada, se utiliza **422 Unprocessable Entity**. Los permisos se manejan mediante **403 Forbidden** y la ausencia de un recurso mediante **404 Not Found**.

Tabla 13: Correspondencia entre violación de regla y respuesta HTTP.

Regla	HTTP	Excepción y significado
RN-02	409	<code>SlotAlreadyBookedException</code> . El bloque solicitado ya se encuentra ocupado por una reserva confirmada. Se utiliza tanto cuando el conflicto se detecta en <code>BookingService</code> como cuando se detecta al guardar por medio de la restricción <code>reserva_sin_solape</code> .
RN-03	403	<code>ForbiddenOperationException</code> . Un usuario final intenta cancelar una reserva que pertenece a otro usuario.
RN-04	409	<code>PastBookingCancellationException</code> . La fecha y hora de inicio de la reserva ya ocurrieron y, por lo tanto, ya no puede cancelarse.
RN-06	409	<code>ActiveBookingLimitExceededException</code> . El usuario alcanzó el tope de reservas activas configurado para el sistema.
RN-07	403	<code>ForbiddenOperationException</code> . Un usuario que no es administrador intenta realizar una operación de gestión sobre las canchas, horarios o bloqueos de mantenimiento.
—	404	<code>NotFoundException</code> . La reserva, la cancha o el usuario solicitado no existe.
—	422	<code>CourtNotBookableException</code> . La cancha existe, pero no admite la reserva porque está inactiva, el bloque ya pasó, se encuentra fuera del horario de atención o coincide con un bloqueo de mantenimiento.
—	400	La petición contiene datos inválidos, no cumple las validaciones definidas para la entrada o contiene un valor que no corresponde con el tipo esperado.

Las **RN-01**, **RN-05** y **RN-08** no generan una respuesta de error propia. La **RN-01** se

controla al construir el bloque horario y mediante las validaciones de entrada. Por su parte, la **RN-05** y la **RN-08** describen el comportamiento y los estados de una reserva, por lo que no representan por sí mismas una operación inválida que deba traducirse a un código HTTP.

7. Resultados

7.1. Funcionalidades implementadas

En la versión actual del proyecto se encuentran implementadas las funcionalidades principales definidas para el usuario final y el administrador. Los recorridos pueden realizarse desde los microfrontends y utilizan los servicios del backend correspondientes para completar cada operación.

- **Reserva y cancelación por parte del usuario final.** Se permite el registro e inicio de sesión, la consulta de disponibilidad por deporte, cancha y fecha, y la selección de un bloque horario libre. Una vez creada la reserva, esta aparece en la sección de reservas del usuario y puede cancelarse mientras todavía no haya comenzado. Al cancelar, el bloque vuelve a mostrarse como disponible.
- **Administración de canchas y reservas.** El administrador puede crear y editar canchas, cambiar su estado entre activa e inactiva y gestionar bloqueos de mantenimiento. También dispone de un listado global de reservas desde el cual puede cancelar reservas de cualquier usuario, siempre que todavía cumplan las condiciones de cancelación establecidas por las reglas de negocio.
- **Reportes de uso.** El módulo de reportes permite seleccionar un rango de fechas y consultar los cuatro indicadores definidos para el proyecto: número de reservas por cancha y deporte, porcentaje de ocupación por cancha, número de cancelaciones y ranking de canchas según su demanda. El panel de administración utiliza estos mismos datos para mostrar un resumen correspondiente al día actual.
- **Gestión de usuarios.** El administrador puede consultar las cuentas registradas y cambiar su estado entre activo e inactivo. Cuando una cuenta se inactiva, sus reservas se conservan. Si el usuario mantenía una sesión abierta, la siguiente petición autenticada es rechazada, debido a que el API Gateway vuelve a validar las credenciales en cada solicitud. Ante este caso, la interfaz elimina la sesión almacenada y devuelve al usuario a la pantalla de inicio de sesión.

El proyecto también incluye la infraestructura necesaria para ejecutar estas funcionalidades de forma conjunta. El archivo `docker-compose.yml` define once servicios: el edge, el

shell, los tres microfrontends remotos, el API Gateway, los cuatro microservicios y PostgreSQL. El sistema puede levantarse mediante `docker compose up`, y las migraciones cargan los datos de demostración utilizados por usuarios, canchas y reservas durante el primer arranque.

7.2. Cumplimiento de los criterios de aceptación

Los criterios de aceptación permiten comprobar si la implementación alcanzó el mínimo funcional y técnico definido para el proyecto. Para esta verificación se consideran las funcionalidades disponibles en los microfrontends, los casos de uso implementados en el backend, las pruebas existentes y la configuración de despliegue.

Tabla 14: Cumplimiento de los criterios de aceptación.

#	Criterio	Estado	Evidencia
CA-1	El usuario final puede consultar disponibilidad, crear y cancelar reservas de pádel, tenis y básquet.	Sí	<code>mf-reservas</code> implementa las pantallas de disponibilidad, nueva reserva y reservas propias. La creación y cancelación se comprueban en <code>BookingServiceCreacionTest</code> y <code>BookingServiceCancelacionTest</code> . El catálogo inicial incluye canchas activas de los tres deportes.
CA-2	El administrador puede gestionar el catálogo de canchas y cancelar reservas del sistema.	Sí	<code>mf-administracion</code> incluye la gestión de canchas y el listado global de reservas. <code>CourtServiceTest</code> comprueba las operaciones administrativas y <code>BookingServiceCancelacionTest</code> la cancelación realizada por el administrador.
CA-3	El sistema impide crear una reserva sobre un bloque horario ya ocupado.	Sí	La validación se realiza en <code>BookingService</code> y se refuerza en PostgreSQL con la restricción <code>reserva_sin_solape</code> . Las pruebas <code>BookingServiceCreacionTest</code> y <code>ConcurrenciaReservaIT</code> cubren el conflicto y el caso concurrente.

Continúa en la siguiente página

Continuación de la Tabla 14

#	Criterio	Estado	Evidencia
CA-4	El módulo de reportes muestra al menos la ocupación por cancha y el número de reservas por período.	Sí	mf-reportes presenta los indicadores por rango de fechas y el panel de administración reutiliza estos mismos datos para mostrar un resumen del día actual. ms-reportes implementa los cuatro indicadores definidos para el proyecto: reservas por período, ocupación por cancha, cancelaciones y ranking de demanda, los cuales también se verifican en <code>ReportServiceTest</code> .
CA-5	El frontend está compuesto por un shell y al menos dos microfrontends remotos integrados mediante Module Federation.	Sí	El shell integra mediante Module Federation a mf-reservas , mf-administracion y mf-reportes , cada uno publicado como un remote independiente.
CA-6	El backend está compuesto por al menos dos microservicios Spring Boot independientes, persistiendo en PostgreSQL.	Sí	El despliegue contiene ms-usuarios , ms-canchas , ms-reservas y ms-reportes . Los tres primeros administran respectivamente usuarios_db , canchas_db y reservas_db en PostgreSQL, mientras que ms-reportes obtiene la información por HTTP sin mantener una base propia.

De acuerdo con estas evidencias, los seis criterios de aceptación definidos para el proyecto se encuentran cubiertos por la implementación actual.

7.3. Verificación de las reglas de negocio

La verificación de las reglas de negocio se apoya principalmente en pruebas automatizadas sobre los servicios de aplicación y se complementa con los recorridos funcionales del sistema. De esta manera, se comprueban tanto los casos válidos como las situaciones que deben ser rechazadas, sin depender únicamente de la interfaz gráfica.

Tabla 15: Comprobación de las reglas de negocio.

Regla	Escenario	Resultado
RN-01	Crear una reserva para una cancha, una fecha futura y un bloque que inicia a las 19:00.	<code>BookingServiceCreacionTest</code> comprueba que la reserva conserve la cancha y la fecha seleccionadas, y que el bloque generado sea de 19:00 a 20:00.
RN-02	Intentar crear una reserva sobre un bloque que ya tiene una reserva confirmada para la misma cancha.	<code>BookingServiceCreacionTest</code> comprueba que la operación sea rechazada con <code>SlotAlreadyBookedException</code> y que la nueva reserva no se guarde.
RN-02	Intentar guardar de forma concurrente dos reservas para la misma cancha, fecha y bloque horario.	<code>ConcurrenciaReservaIT</code> comprueba que solo una operación consiga guardar la reserva y que la otra sea rechazada con <code>SlotAlreadyBookedException</code> . La prueba está configurada con diez repeticiones.
RN-03	Un usuario final intenta cancelar una reserva perteneciente a otro usuario y un administrador intenta cancelar esa misma reserva.	<code>BookingServiceCancelacionTest</code> comprueba que el usuario final sea rechazado con <code>ForbiddenOperationException</code> y que el administrador pueda cambiar la reserva a <code>CANCELADA</code> .
RN-04	Intentar cancelar una reserva cuya hora de inicio ya ocurrió.	<code>BookingServiceCancelacionTest</code> comprueba que la cancelación sea rechazada con <code>PastBookingCancellationException</code> , tanto para un usuario final como para un administrador.
RN-05	Cancelar una reserva confirmada que todavía no ha comenzado.	<code>BookingServiceCancelacionTest</code> comprueba que la reserva se guarde como <code>CANCELADA</code> y que se registre la fecha y hora de cancelación. Al dejar de estar confirmada, deja de ocupar el bloque horario.

Continúa en la siguiente página

Continuación de la Tabla 15

Regla	Escenario	Resultado
RN-06	Intentar crear una nueva reserva cuando el usuario ya tiene tres reservas activas y el límite configurado es 3.	<code>BookingServiceCreacionTest</code> comprueba que la operación sea rechazada con <code>ActiveBookingLimitExceededException</code> y que no se guarde una nueva reserva.
RN-07	Un usuario final intenta crear o modificar una cancha, mientras que un administrador realiza una operación de creación.	<code>CourtServiceTest</code> comprueba que las operaciones de gestión sean rechazadas para el usuario final y permitidas para el administrador.
RN-08	Crear, cancelar y consultar reservas cuyo bloque ya terminó.	Las pruebas comprueban que una reserva nueva sea CONFIRMADA , que una cancelación la deje en CANCELADA y que las reservas cuyo bloque ya terminó dejen de contarse como activas al obtener su estado vigente.

Para el caso concurrente de la **RN-02**, la comprobación se realiza contra PostgreSQL y no únicamente con dependencias simuladas. La prueba `ConcurrenciaReservaIT` se habilita cuando se proporciona la conexión mediante `RESERVAS\_IT\_DB\_URL`, ya que necesita utilizar la restricción real de la base de datos.

7.4. Limitaciones

Aunque las funcionalidades definidas en el alcance están implementadas, la versión actual del sistema todavía presenta algunas limitaciones:

- **Los listados no tienen paginación.** Los endpoints de usuarios, canchas y reservas devuelven las colecciones completas. Para el volumen de datos utilizado en el proyecto esto no genera problemas, pero si la cantidad de registros aumenta sería necesario incorporar paginación.
- **Parte de los reportes se calcula en memoria.** `ms-reportes` obtiene la información desde `ms-reservas` y `ms-canchas` mediante sus APIs y realiza los conteos y agrupaciones en `ReportService`. Con este enfoque se mantiene la separación de datos entre los microservicios, aunque el servicio debe obtener y procesar los registros necesarios para cada consulta.

- **No se mantiene un historial de activación de las canchas.** Para calcular la ocupación de un período se utiliza el horario actual de cada cancha. Si una cancha estuvo inactiva durante una parte del rango consultado, esos días no pueden identificarse de forma independiente, porque `canchas_db` conserva únicamente su estado actual.
- **Algunas pruebas necesitan PostgreSQL disponible.** Las pruebas de arranque de `ms-usuarios`, `ms-canchas` y `ms-reservas` utilizan Testcontainers, por lo que requieren Docker para ejecutar PostgreSQL durante la prueba. Además, `ConcurrenciaReservaIT` necesita una instancia de PostgreSQL configurada mediante `RESERVAS\_IT\_DB\_URL`. Las pruebas unitarias de los servicios de aplicación pueden ejecutarse sin levantar esta infraestructura.

8. Conclusiones

El objetivo general del proyecto se cumplió mediante la implementación de un sistema web que permite consultar disponibilidad, crear y cancelar reservas, gestionar canchas y usuarios, y consultar reportes básicos. Estas funcionalidades se encuentran distribuidas entre los distintos módulos del frontend y los servicios del backend, y los seis criterios de aceptación definidos para el proyecto cuentan con evidencia dentro de la implementación.

La separación del frontend mediante un `shell` y tres microfrontends permitió organizar las funcionalidades de reservas, administración y reportes en módulos diferenciados, manteniendo su integración mediante Module Federation. De forma similar, el backend se dividió en `ms-usuarios`, `ms-canchas`, `ms-reservas` y `ms-reportes`, de manera que cada servicio concentra las responsabilidades relacionadas con su área funcional.

La estrategia de persistencia permitió mantener separados los datos administrados por cada microservicio. `ms-usuarios`, `ms-canchas` y `ms-reservas` trabajan con bases de datos independientes dentro de una misma instancia de PostgreSQL y no acceden directamente a las tablas pertenecientes a otros servicios. Cuando se necesita información de otro dominio, esta se obtiene mediante la API del microservicio responsable.

Las reglas de negocio definidas para el proceso de reserva también fueron implementadas y verificadas. En particular, el control de solapamiento permitió comprobar que una validación realizada únicamente antes de guardar una reserva no es suficiente cuando existen solicitudes concurrentes. Por esta razón, la comprobación realizada en `ms-reservas` se complementó con una restricción en PostgreSQL, evitando que se persistan dos reservas confirmadas sobre el mismo horario.

Finalmente, las decisiones arquitectónicas permitieron mantener una separación clara

entre la lógica de negocio, la comunicación, la persistencia y la capa de presentación. La implementación también permitió identificar límites del diseño actual, como la ausencia de paginación, el procesamiento en memoria de parte de los reportes y la falta de un historial de activación de las canchas. Estas limitaciones no impiden cumplir el alcance definido, pero permiten reconocer aspectos que deberían revisarse si el sistema necesitara manejar un escenario de mayor tamaño.

8.1. Lecciones aprendidas

Uno de los principales aprendizajes que deja el proyecto está relacionado con el control de reglas de negocio cuando existen operaciones concurrentes. En el caso del solapamiento de reservas, la comprobación realizada por `BookingService` antes de guardar no cubre por sí sola el escenario en el que dos solicitudes intentan reservar el mismo bloque al mismo tiempo. Por esta razón, la validación se complementa con la restricción `reserva_sin_solape` en PostgreSQL. Si el diseño se planteara nuevamente, este tipo de condiciones de concurrencia se consideraría desde el inicio junto con las reglas de negocio y el modelo de persistencia.

Otro aprendizaje se relaciona con la separación de datos entre microservicios. Mantener una base de datos independiente para cada servicio evita el acceso directo a información administrada por otro dominio, pero también hace necesario definir las comunicaciones entre servicios. Esto ocurre en `ms-reservas`, que consulta información de las canchas mediante `ms-canchas`, y en `ms-reportes`, que necesita información de `ms-reservas` y `ms-canchas`. En un nuevo diseño, estas necesidades de información y los contratos entre servicios se definirían desde etapas tempranas de la arquitectura.

En el frontend, el uso de microfrontends permite mantener separadas las funcionalidades de reservas, administración y reportes, aunque también requiere coordinar la integración entre el `shell` y los remotes, así como la configuración de Module Federation y las dependencias compartidas. Si se volviera a desarrollar la aplicación, estas configuraciones y convenciones comunes se definirían desde el inicio para mantener una misma estructura entre los distintos microfrontends.

Finalmente, el módulo de reportes permitió identificar una necesidad que el modelo actual no cubre: conservar información histórica sobre los cambios de estado de las canchas. Actualmente se mantiene únicamente su estado presente, por lo que no es posible determinar si una cancha estuvo inactiva durante una parte de un período consultado. Si se necesitara mayor precisión en los reportes históricos, este comportamiento debería considerarse desde el diseño del modelo de datos y almacenar los cambios de estado necesarios.

8.2. Trabajo futuro

A partir de las limitaciones identificadas durante la implementación, una evolución del sistema podría incorporar paginación en los listados de usuarios, canchas y reservas. También se podría revisar la forma en que se generan los reportes, ya que actualmente parte de los cálculos se realiza en memoria a partir de la información obtenida desde ms-reservas y ms-canchas. En caso de manejar un mayor volumen de información, este proceso debería adaptarse para evitar obtener y procesar colecciones completas en cada consulta.

Estas funcionalidades se plantean como posibles extensiones del sistema y no forman parte de la implementación desarrollada en el alcance actual del proyecto.

Referencias

- [1] S. Brown. «The C4 model for visualising software architecture,» visitado 28 de ago. de 2026. dirección: <https://c4model.com/>
- [2] Docker Inc. «Docker Compose Documentation,» visitado 28 de ago. de 2026. dirección: <https://docs.docker.com/compose/>
- [3] Module Federation. «Module Federation Documentation,» visitado 28 de ago. de 2026. dirección: <https://module-federation.io/>
- [4] PostgreSQL Global Development Group. «PostgreSQL Documentation,» visitado 28 de ago. de 2026. dirección: <https://www.postgresql.org/docs/>
- [5] Spring. «Spring Boot Reference Documentation,» visitado 28 de ago. de 2026. dirección: <https://docs.spring.io/spring-boot/documentation.html>

A. Scripts de base de datos

Los scripts completos se mantienen versionados junto con la implementación. Las siguientes rutas permiten consultar la versión exacta aplicada por cada microservicio, incluidos los datos semilla:

- Inicialización de PostgreSQL: <https://github.com/miguesj-hub/reserva-canchas/tree/main/infra/postgres/init>
- Migraciones de usuarios: <https://github.com/miguesj-hub/reserva-canchas/tree/main/backend/ms-usuarios/src/main/resources/db/migration>
- Migraciones de canchas: <https://github.com/miguesj-hub/reserva-canchas/tree/main/backend/ms-canchas/src/main/resources/db/migration>
- Migraciones de reservas: <https://github.com/miguesj-hub/reserva-canchas/tree/main/backend/ms-reservas/src/main/resources/db/migration>

B. Colección de pruebas de la API

La colección Postman contiene las peticiones de prueba que atraviesan el API Gateway, incluidas las variables de conexión, autenticación y los endpoints de usuarios, canchas, reservas y reportes:

https://github.com/miguesj-hub/reserva-canchas/blob/main/Postman-collection/reserva-canchas.postman_collection.json

C. Capturas de la aplicación

D. Repositorio del proyecto

La implementación, las migraciones de base de datos y la colección de pruebas Postman se encuentran disponibles en el siguiente repositorio:

<https://github.com/miguesj-hub/reserva-canchas>